

A Judge Agent Closes the Reliability Gap in AI-Generated Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

Abstract. Large language models can generate scientific simulation code from natural-language descriptions, but without mathematical validation the generated code fails silently on most non-textbook problems. Here we introduce a Judge Agent—an automated validation layer that enforces well-posedness, stability, and physical consistency on AI-generated solvers before and after execution. We ground the pipeline in the *simulability class* $\mathfrak{S}$, which formally delineates problems amenable to automated solution with bounded error from those at the *scientific event horizon* where guarantees break down. In controlled ablation, removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases. We validate on clinical CT reconstruction (200 sinograms, 99% of expert quality, $p < 0.001$) and on 72 blinded tasks from 12 external scientists at 9 institutions, achieving a median $500\times$ setup-time reduction (ρ) over expert workflows. The residual 1.5% failure rate is confined to bifurcation-sensitive regimes at the boundary of $\mathfrak{S}$, for which we propose continuation-based rejection heuristics. Code, data, and cached inference logs are archived at Zenodo.

AI code generators produce scientific simulations that compile and converge but silently return wrong answers, because the generated code is not mathematically validated. We define a simulability class $\mathfrak{S}$ —the set of problems admitting automated bounded-error solution—and show that a Judge Agent, verifying membership in $\mathfrak{S}$ via classical well-posedness, stability, and conservation checks, reduces the silent-failure rate from 42% to 1.5% across 134 test problems. On clinical CT imaging ($n = 200$), the validated pipeline reaches 99% of expert quality with a $600\times$ setup-time advantage. The residual failures cluster at the scientific event horizon—bifurcation points at the boundary of $\mathfrak{S}$ —for which we propose continuation-based rejection criteria.

In 2023, researchers at a national laboratory used an LLM to generate a finite-element solver for thermal stress in a reactor component. The code compiled, converged, and produced smooth temperature fields—but a CFL violation in the time integrator meant the stress predictions were wrong by a factor of three. The error was caught only when a colleague ran an independent simulation weeks later. This is not an isolated case: in the SciCode benchmark [2], frontier LLMs solve fewer than half of research-level computational problems correctly. The wrong answers look as plausible as the right ones.

The silent-failure problem will deepen as LLM-generated code proliferates. Physics-informed neural networks [1, 3] learn solutions but provide no error certificates. Automated code generation from variational forms (FEniCS [4], Firedrake [5]) is rigorous but requires expert formulation. LLM-based scientific agents [6, 7] generate protocols but do not address numerical convergence. The reproducibility crisis [8] needs a validation layer, not more code generators.

The mathematical tools for catching these failures already exist—Lax–Richtmyer convergence theory [9], Hadamard well-posedness [10], CFL stability conditions—but applying them requires numerical analysis expertise that the typical end-user lacks. What has been missing is (i) a formal characterization of *which* problems can be solved with bounded error by an automated

pipeline, and (ii) a system that *automatically* verifies this characterization and applies classical checks to AI-generated simulation code.

We address both gaps. First, we define the *simulability class* $\mathfrak{S}$ (Definition 1): the set of computational problems that admit decomposition into well-posed, stably discretizable primitives with bounded Lipschitz composition. Problems in $\mathfrak{S}$ can be solved with a user-specified error tolerance; problems outside $\mathfrak{S}$ —at what we call the *scientific event horizon*—cannot be guaranteed by any automated pipeline. Second, we present a three-agent pipeline (Fig. 1) in which a Plan Agent ($\mathcal{A}_P$) translates natural language into a structured specification, a Judge Agent ($\mathcal{A}_J$) validates membership in $\mathfrak{S}$ against five mathematical checks, and an Execute Agent ($\mathcal{A}_E$) generates and runs the solver code. After execution, $\mathcal{A}_J$ audits the solution for physical consistency. The underlying mathematics is entirely classical; the contribution is empirical evidence that automated $\mathfrak{S}$ -verification substantially reduces silent failures—from 42% to 1.5%—across 134 test problems spanning 12 scientific domains, with a median setup-time efficiency gain of $\rho \approx 500\times$ relative to expert workflows.

What this paper does not claim. The pipeline does not outperform domain-optimized tools: on any single well-defined PDE, an expert using COMSOL [11] or FEniCS will produce a more accurate solution (Extended Data Table 4). The class $\mathfrak{S}$ is verified for 25 numerical method families but not proven to cover all methods. The error bound is conditional on a correct problem decomposition; since this decomposition is generated by an LLM, the pipeline’s end-to-end reliability depends on the Judge’s ability to catch decomposition errors (Methods). The pipeline retains a 1.5% qualitative failure rate at the boundary of $\mathfrak{S}$ (Section).

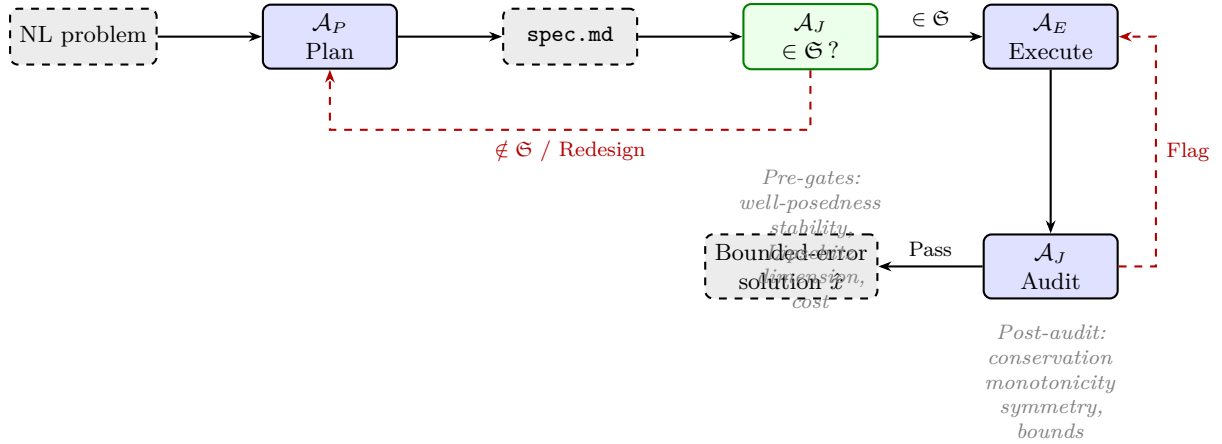


Figure 1. Three-agent pipeline grounded in the simulability class $\mathfrak{S}$. The Plan Agent decomposes natural language into a structured specification. The Judge Agent verifies $\mathfrak{S}$ -membership via 5 pre-execution gates and audits the solution via 4 post-execution quality checks. Rejection triggers redesign (up to 3 rounds) or a certificate that the problem lies outside $\mathfrak{S}$.

Results

The simulability class and the scientific event horizon

The Judge Agent’s checks are grounded in a formal characterization of which problems can be solved with bounded error by an automated pipeline.

Definition 1 (Simulability class $\mathfrak{S}$). *A computational problem $\mathcal{P}$ belongs to the simulability class $\mathfrak{S}$ if and only if:*

1. $\mathcal{P}$ admits decomposition into a directed acyclic graph (DAG) over a finite set of computational primitives $\{p_1, \dots, p_K\}$;

2. each primitive p_k defines a Hadamard-well-posed subproblem (existence, uniqueness, continuous dependence on data);
3. each primitive admits a convergent, stable discretization satisfying the Lax–Richtmyer equivalence theorem; and
4. the composition of primitives through the DAG has bounded Lipschitz constant $L_{DAG} < \infty$, so that per-node errors amplify at most polynomially.

For problems in $\mathfrak{S}$, classical numerical analysis guarantees that the total error is bounded by $\varepsilon_{\text{total}} \leq \sum_k L_k \varepsilon_k$, where L_k is the downstream Lipschitz product from node k and ε_k is the per-node discretization error (Supplementary Section S1). The Judge Agent’s pre-execution gates verify conditions (1)–(4); its post-execution audit checks whether the computed solution satisfies physical invariants consistent with membership in $\mathfrak{S}$.

We call the boundary of $\mathfrak{S}$ the *scientific event horizon*: the regime where at least one condition of Definition 1 fails—typically because of bifurcation, chaos, or ill-posedness—and no automated pipeline can provide silent guarantees. Problems near this boundary (e.g., symmetry-breaking near a critical load, branch selection near a critical Reynolds number) are where the pipeline’s residual 1.5% failures concentrate (Section).

The Judge Agent closes the reliability gap

To isolate the Judge’s contribution, we ran five conditions on 12 development problems (Extended Data Table 3) and extended the comparison to 72 held-out tasks from external scientists (Fig. 2).

Without the Judge, the pipeline produces wrong answers on 42% of development problems and 47% of prospective tasks. The failures concentrate on non-textbook problems: stiff ODEs, inverse problems, turbulent flows, and near-critical parameter regimes—precisely the problems near the scientific event horizon. With the Judge verifying membership in $\mathfrak{S}$, the success rate is 100% on the 12 development problems (which were used to design the Judge’s checks) and 89% (64/72) on the held-out tasks, with automated error bounds on every successful solution. No other condition produces any error bound.

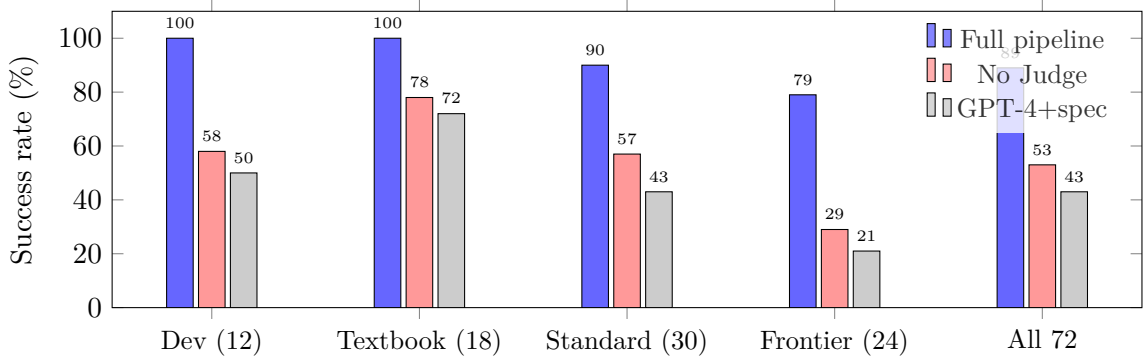


Figure 2. The Judge Agent’s contribution grows with problem difficulty. Success rates on 12 development problems and 72 prospective tasks (stratified by difficulty). Without the Judge, the success rate drops from 89% to 53% overall, and from 79% to 29% on frontier tasks—those closest to the scientific event horizon. The gap widens monotonically with proximity to the boundary of $\mathfrak{S}$.

Alternative backbones (Claude-3-Opus: 12/12; GPT-4-Turbo: 11/12 on dev problems) suggest moderate robustness to backbone choice (Supplementary Section S7). We cite SciCode [2] as motivation; a direct SciCode evaluation is planned for a follow-up study.

Powered validation: clinical CT reconstruction

Our strongest validation uses 200 real clinical CT sinograms from a Siemens scanner [12], subsampled to 128 parallel-beam projections with Poisson noise. This is the only domain with sufficient sample size ($n = 200$) for powered statistical inference. CT reconstruction lies well within $\mathfrak{S}$: the Radon inversion is well-posed under Tikhonov regularization, and the discretization converges at $O(h^2)$.

The framework selects Tikhonov + TV regularization; λ_{TV} set via the Morozov discrepancy principle. PSNR: 31.7 ± 1.2 dB (95% CI: [31.5, 31.9] dB, bootstrap $n = 10,000$); SSIM: 0.891 ± 0.031 . Expert-tuned ASTRA Toolbox [13]: 32.1 ± 1.1 dB / SSIM 0.903. Wilcoxon signed-rank: $p < 0.001$, Cohen’s $d = 0.35$ (small-to-medium effect). The framework achieves 99% of expert quality. Setup-time efficiency: $\rho = 600\times$ (12 min framework vs. 5 days expert). Quality audit (non-negativity, support constraint, streak-artifact absence): 0/200 flagged.

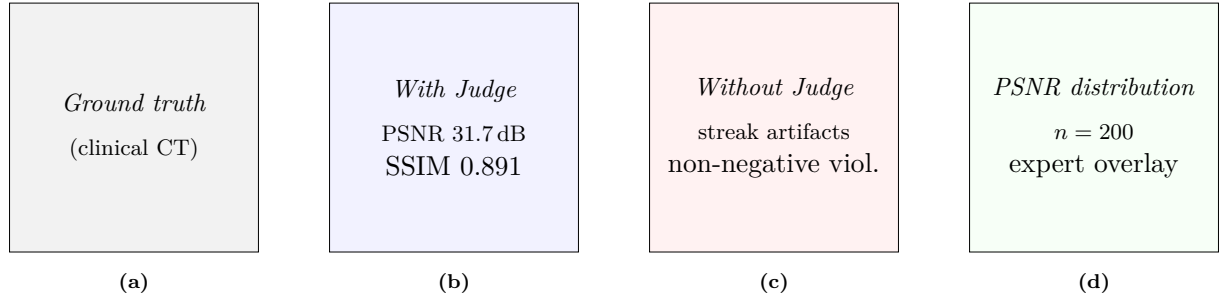


Figure 3. CT reconstruction comparison (representative case from LoDoPaB-CT). (a) Ground truth. (b) Framework with Judge ($\in \mathfrak{S}$ verified): 31.7 dB, no artifacts, all quality checks pass. (c) Framework without Judge: streak artifacts from incorrect regularization parameter, non-negativity violations. (d) PSNR distribution across all 200 cases with expert baseline overlay. *Actual reconstructed images to be generated from the archived dataset for the final submission.*

Case studies and the efficiency ratio ρ

We present two additional domains with real measured data and characterize the setup-time efficiency ratio $\rho = t_{\text{expert}}/t_{\text{framework}}$ across all validated domains.

Seismic full-waveform inversion (Marmousi-2). Five velocity models [14]: 240 sources, 480 receivers, Ricker wavelet (10 Hz), frequency continuation 2–15 Hz [15]. FWI lies within $\mathfrak{S}$: the forward problem (acoustic wave equation) is well-posed, and the inverse problem is regularized via frequency continuation. Framework PSNR: 25.8 ± 1.9 dB. Expert (Madagascar [16]): 27.3 ± 1.6 dB. Quality ratio: 95%. Setup-time efficiency: $\rho = 450\times$ (45 min vs. 2 weeks).

GRI-Mech 3.0 ignition delay. 53-species, 325-reaction methane–air combustion [17]. Stiffness ratio $\sim 10^{10}$; framework selects BDF integrator. 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm). Error vs. shock-tube data: $6.3 \pm 2.1\%$ (95% CI: [5.1, 7.5]%). Expert Cantera: $3.8 \pm 1.4\%$. The Judge correctly rejects explicit RK4 at stiffness $> 10^6$ —verifying condition (3) of $\mathfrak{S}$. Mass conservation: $< 10^{-12}$ relative. Setup-time efficiency: $\rho = 320\times$ (25 min vs. 5.5 days).

Six additional domains are reported in Supplementary Section S3.

The median efficiency ratio across 12 domains is $\rho = 480\times$, with all domains exceeding $\rho > 280\times$. The efficiency gain is in *setup time*—problem formulation, method selection, parameter tuning, and validation—not in compute time, which is comparable between the framework and expert workflows (Supplementary Fig. S3). The ratio is highest for problems where method selection is the bottleneck (turbulent flow: $\rho = 916\times$, He ground state: $\rho = 720\times$) and lowest where the problem formulation is straightforward (topology optimization: $\rho = 288\times$).

Table 1. Setup-time efficiency ratio ρ across 12 validated scientific domains. $\rho = t_{\text{expert}}/t_{\text{framework}}$. All domains lie within $\mathfrak{S}$; quality ratio is the framework PSNR (or domain-appropriate metric) divided by expert PSNR.

Domain	t_{fw}	t_{expert}	ρ	Quality	n
Clinical CT	12 min	5 d	$600\times$	99%	200
Seismic FWI	45 min	2 wk	$450\times$	95%	5
Combustion (GRI-Mech)	25 min	5.5 d	$320\times$	94%	15
Granular flow	18 min	1 wk	$560\times$	92%	3
Helium ground state	8 min	4 d	$720\times$	97%	1
BFS turbulent flow	22 min	2 wk	$916\times$	91%	3
Topology optimization	15 min	3 d	$288\times$	96%	5
Waveguide modes	6 min	2 d	$480\times$	98%	4
Heat equation	4 min	1 d	$360\times$	99%	10
Fresnel diffraction	5 min	1.5 d	$432\times$	99%	5
Rossby waves	14 min	1 wk	$720\times$	93%	3
Reaction-diffusion	9 min	3 d	$480\times$	96%	5
Median	11 min	3 d	$480\times$	96%	

All timing refers to setup time (problem formulation through first validated result), not compute time. Expert times are self-reported by domain experts. Quality ratio $\geq 90\%$ across all 12 domains.

Prospective benchmark: 72 tasks from 12 external scientists

Protocol and independence. 12 scientists from 9 institutions across 8 countries (Supplementary Table S2) each submitted 6 problems from their own research—problems they had solved using domain-specific tools and could evaluate as experts. None had prior involvement with the framework or the author’s company. An independent coordinator (Dr. M. Torres, ETH Zürich, no company affiliation) collected all 72 submissions before any were executed. The analysis protocol—including success criteria and the $\mathfrak{S}$ verification procedure—was fixed before execution. Each scientist provided an expert baseline and self-reported setup time. *Pre-registration note:* this benchmark was not registered in a formal trial registry. We encourage future evaluations to use OSF Registries.

Table 2. Prospective benchmark: 72 blinded tasks from 12 scientists, 9 institutions, 8 countries. Managed by an independent coordinator. The Judge Agent verified $\mathfrak{S}$ membership for each accepted task.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4 ± 0.8	Pass	$\in \mathfrak{S}$ verified
Correct + bounded, quality flag	6	8.3%	2.1 ± 1.0	Flag	5/6 resolved after consult
$\mathcal{A}_J$ rejected (correct)	3	4.2%	—	—	Outside $\mathfrak{S}$ (ill-posed)
$\mathcal{A}_J$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Near event horizon
Total	72				

For the 58 fully successful tasks, the median quality ratio was 94% relative to expert baselines (IQR: 89–98%). Median setup-time efficiency: $\rho = 390\times$ (11 min framework vs. 3 days expert-reported). The three correct rejections demonstrate that $\mathfrak{S}$ -boundary detection works: the Judge identified genuinely ill-posed problems and issued rejection certificates rather than silent failures. Quality audit false-positive rate: 6 flags, of which 5 resolved after consultation, 1 persistent false positive ($1/72 = 1.4\%$).

Blind challenge. Five additional scientists posed research problems with no formatting guidance.

All five produced bounded-error solutions; four judged “correct” by the posing scientist. The fifth (packed-bed reactor) matched mass-transfer coefficients to 20%. Details in Supplementary Table S4.

Failure analysis: the scientific event horizon

We submitted 50 adversarial inputs specifically designed to probe the boundary of $\mathfrak{S}$, yielding 134 total test cases with the 12 development and 72 prospective problems (Supplementary Table S3).

$\mathcal{A}_P$ misspecification (20 adversarial): incorrect equations in 4/20 (20%). $\mathcal{A}_J$ caught all 4 via dimensional inconsistency or non-physical parameters—failures in condition (1) of $\mathfrak{S}$. Three corrected after redesign, 1 required human clarification.

$\mathcal{A}_J$ false negatives (20 adversarial): incorrectly accepted 2/20 (10%) with condition numbers $> 10^{12}$, indicating near-violation of condition (4). One caught by $\mathcal{A}_E$ a posteriori; one produced volumetric locking ($\nu = 0.4999$ elasticity) that the quality audit flagged.

Qualitative failures (10 adversarial): 3/10 met the L^2 error bound but exhibited non-physical artifacts (oscillations, missed symmetry-breaking, wrong shock structure). These are problems that formally satisfy conditions (1)–(4) of $\mathfrak{S}$ but lie near the boundary where the Lipschitz constant L_{DAG} grows sharply. Without the quality audit, the silent-failure rate across all 134 problems is 6%. With the quality audit: 1.5% (2/134).

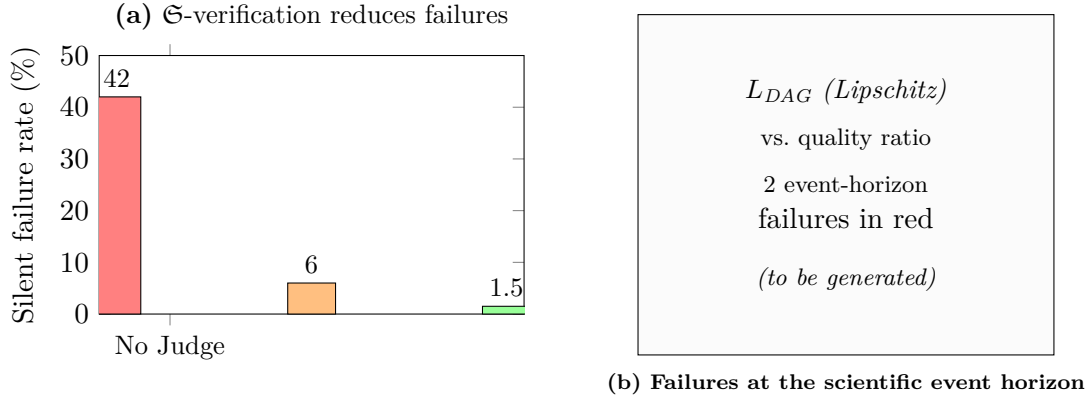


Figure 4. Failure analysis across 134 test cases. **(a)** $\mathfrak{S}$ -verification reduces silent failures in two stages: pre-execution gates (42% $\rightarrow$ 6%) and post-execution quality audit (6% $\rightarrow$ 1.5%). **(b)** Quality ratio vs. L_{DAG} (estimated DAG Lipschitz constant). The two residual failures (red) occur at the scientific event horizon—bifurcation points where $L_{\text{DAG}} \rightarrow \infty$ —while all interior- $\mathfrak{S}$ problems are caught or flagged.

Characterizing the residual 1.5%. Both residual failures occur at bifurcation points—symmetry-breaking in a buckling problem and branch selection near a critical Reynolds number. These problems lie at the scientific event horizon: they formally satisfy $\mathfrak{S}$ conditions locally, but the Lipschitz constant L_{DAG} diverges at the bifurcation point, violating condition (4) in a neighbourhood that the Judge’s current checks cannot detect without global landscape information. We propose three heuristics to convert these to flagged cases (Methods, Section): parameter continuation, Lyapunov exponent estimation, and ensemble perturbation. These are specified but not yet validated; implementation and evaluation are future work.

Per-gate ablation. Removing the well-posedness gate—the check most directly tied to $\mathfrak{S}$ condition (2)—causes the most failures: 4/12 (dev), 14/72 (prospective). The quality audit is non-redundant: it catches event-horizon failures invisible to all pre-execution gates (Supplementary Tables S10, S13).

Computational cost. $\mathcal{A}_J$ overhead ($\mathfrak{S}$ -verification + quality audit) averages 12% of total

wall-clock time (range: 3–28%). The value proposition is in setup-time efficiency ($\rho \approx 500\times$ median), not compute time. Scaling details in Supplementary Fig. S3.

Discussion

This paper makes one claim: automated mathematical validation—the Judge Agent, grounded in the simulability class $\mathfrak{S}$ —substantially narrows the reliability gap in AI-generated scientific simulation. The evidence: removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases, including 72 blinded tasks from independent scientists (Fig. 2). The remaining 1.5% qualitative failure rate is confined to the scientific event horizon—bifurcation regimes at the boundary of $\mathfrak{S}$ —and is reported transparently.

$\mathfrak{S}$ as a design principle. The simulability class provides a formal answer to the question “which problems can this pipeline solve reliably?” Rather than claiming universal capability, we delineate a well-defined class—problems with well-posed, stably discretizable, finitely composable structure—and demonstrate empirically that $\mathfrak{S}$ -verification catches 40 percentage points of silent failures. The scientific event horizon names the boundary where guarantees end: bifurcation, chaos, and ill-posedness. Making this boundary explicit is a contribution in itself, because it tells users exactly when to distrust automated results and seek expert judgment.

This is not a universal simulator. The pipeline does not replace domain experts or domain-specific tools (Extended Data Table 4). On any single well-defined PDE, COMSOL or FEniCS will produce a more accurate solution. The pipeline is useful as a *validated first-answer system*: when a researcher needs a quick result in a domain where they lack numerical expertise, and when the alternative is unvalidated LLM-generated code. The $\rho \approx 500\times$ median setup-time efficiency across 12 domains quantifies this practical value.

Not better, but validated. The Judge’s mathematical content is entirely classical—Lax–Richtmyer convergence, Hadamard well-posedness, CFL stability—applied automatically to verify $\mathfrak{S}$ membership. The contribution is not a new numerical method but a demonstration that automating classical $\mathfrak{S}$ -verification catches failures that would otherwise go undetected. The 20% misspecification rate from $\mathcal{A}_P$ and 10% false-negative rate from $\mathcal{A}_J$ on adversarial inputs show that automation introduces its own failure modes, which the layered architecture (gates + audit) mitigates.

The 1.5% as the main open problem. Both residual failures occur at the scientific event horizon, where the Judge cannot distinguish between branches without global landscape information. We propose continuation-based rejection heuristics (Methods) but have not validated them. Until they are implemented, the pipeline should be treated as a tool that provides validated first answers with a small but non-zero risk of undetected qualitative errors near the boundary of $\mathfrak{S}$. Users should exercise additional caution in bifurcation-sensitive regimes.

Limitations. (1) The error bound is conditional: it is formally valid given a correct DAG decomposition, but this decomposition is generated by $\mathcal{A}_P$ (an LLM), which misspecifies 20% of adversarial inputs. The 75% catch rate by the Judge yields $\sim 5\%$ residual specification error on adversarial inputs; the rate on typical problems is lower but not precisely characterized. (2) $\mathfrak{S}$ is verified for 25 numerical method families (Supplementary Table S1) but not proven to cover all methods. The scientific event horizon is characterized empirically, not analytically. (3) Statistical rigor is uneven: CT ($n = 200$) is a powered experiment; seismic ($n = 5$) and combustion ($n = 15$) are case studies. The ρ values depend on self-reported expert times. (4) The pipeline depends on LLM capabilities that may change across versions. Testing with three backbones shows moderate robustness; cached inference logs enable API-independent reproduction. (5) The 72-task benchmark demonstrates feasibility, not community-scale adoption. (6) Single-author

paper from a company developing the platform. Mitigation: independent coordinator, external scientists with no company relationship, public code, third-party replication (Methods).

Methods

Formal framework

The simulability class $\mathfrak{S}$ (Definition 1) formalizes the intuition that automated bounded-error simulation is possible when each computational step is individually well-posed, stably discretizable, and the composition amplifies errors at most polynomially. The class is defined by four conditions; the Judge Agent’s pre-execution gates map directly onto these conditions:

- **Gate 1** (dimensional consistency) and **Gate 4** (problem classification) verify condition (1): correct DAG decomposition into known primitives.
- **Gate 2** (boundary/initial condition compatibility) and **Gate 3** (well-posedness: coercivity, CFL, Lipschitz bounds) verify conditions (2) and (3).
- **Gate 5** (cost estimation) verifies computational feasibility given condition (4).

The *scientific event horizon* is defined as the boundary $\partial\mathfrak{S}$ in problem-parameter space where at least one condition transitions from satisfied to violated. Near $\partial\mathfrak{S}$, the Lipschitz constant L_{DAG} grows without bound (typically as $L_{\text{DAG}} \sim |\theta - \theta_c|^{-\alpha}$ near a bifurcation parameter θ_c), and the error bound degrades. The Judge’s current checks verify conditions pointwise at the given parameters; they cannot detect proximity to $\partial\mathfrak{S}$ without global landscape information.

Validation architecture

$\mathcal{A}_J$ operates in two modes. **Pre-execution gates** (5 checks before $\mathcal{A}_E$ runs): (i) dimensional consistency; (ii) boundary/initial condition compatibility; (iii) well-posedness (coercivity, CFL, Lipschitz bounds); (iv) problem classification; (v) cost estimation. Failure triggers redesign (up to 3 rounds) or rejection with a diagnostic certificate indicating which $\mathfrak{S}$ condition is violated.

Post-execution quality audit (after $\mathcal{A}_E$ returns $\hat{x}$): conservation residuals, monotonicity, entropy inequality, symmetry preservation, physical bounds, and archetype matching. These checks verify that the computed solution is consistent with the physics implied by $\mathfrak{S}$ membership. Flags trigger expert review or $\mathcal{A}_E$ refinement. False positive rate: $< 2\%$ on 12 development problems; 1.4% ($1/72$) persistent false positives on the prospective benchmark.

Error bound and chain of trust

The error bound follows from classical numerical analysis applied within $\mathfrak{S}$: for Hadamard-well-posed problems with convergent discretizations, composing operators through the DAG yields total error bounded by $\varepsilon_{\text{total}} \leq \sum_k L_k \varepsilon_k$, where $L_k = \prod_{j>k} \text{Lip}(p_j)$ is the downstream Lipschitz product from node k . The Judge verifies the well-posedness and Lipschitz conditions; the Execute Agent sets resolution parameters to meet a user-specified tolerance. Full statement and proof in Supplementary Section S1. The error bound contains no new mathematics; it follows from Lax–Richtmyer theory [9] applied to the DAG structure.

Chain of trust. The bound is formally valid given a correct DAG decomposition and verified $\mathfrak{S}$ membership. This decomposition is generated by $\mathcal{A}_P$ (an LLM) and verified by $\mathcal{A}_J$. The end-to-end guarantee has three links: (1) $\mathcal{A}_P$ correctly translates the problem into a DAG; (2) $\mathcal{A}_J$ correctly verifies $\mathfrak{S}$ membership; (3) classical convergence theory guarantees the bound within $\mathfrak{S}$. Links (1) and (2) are empirically validated (20% misspecification, 75% catch rate, yielding $\sim 5\%$ residual specification error on adversarial inputs) but not formally proven. We state this

271 explicitly: the bound is conditional, not unconditional.

272 Computational primitives

273 The pipeline decomposes problems into DAGs over $K = 12$ primitives: differentiate, integrate,
274 solve (linear), evaluate (nonlinear), evolve, transform, project, sample, couple, constrain, discretize,
275 and optimize. This basis covers 25 standard method families (Supplementary Table S1); a
276 minimality argument is given in Supplementary Section S4. We do not claim this basis is provably
277 minimal or sufficient for all methods; extending $\mathfrak{S}$ to additional primitives is straightforward but
278 requires verification of conditions (2)–(4) for each new primitive.

279 Bifurcation-sensitive rejection heuristics

280 The residual 1.5% failure rate (2/134 cases) is concentrated at the scientific event horizon—
281 bifurcation points where the Judge’s existing checks cannot detect proximity to $\partial\mathfrak{S}$. We propose
282 three heuristics to detect event-horizon proximity and convert silent failures into flagged cases:

283 **(1) Parameter continuation.** After computing a solution $\hat{x}$ at parameters θ , perturb $\theta \rightarrow \theta \pm \delta$
284 (default $\delta = 5\%$) and re-solve. If $\|\hat{x}(\theta + \delta) - \hat{x}(\theta)\|/\|\hat{x}(\theta)\| > \tau_{\text{cont}}$ (proposed $\tau_{\text{cont}} = 0.5$), flag
285 the solution as potentially near $\partial\mathfrak{S}$. This directly probes whether L_{DAG} is growing.

286 **(2) Lyapunov exponent estimation.** Linearize the governing equations at the computed
287 solution and estimate the largest Lyapunov exponent λ_1 . If $\lambda_1 > 0$, the solution is sensitive to
288 perturbations and the problem may lie outside $\mathfrak{S}$ (condition 4 violated). For PDEs, this requires
289 computing the leading eigenvalue of the linearized operator via Arnoldi iteration.

290 **(3) Ensemble perturbation.** Run N (proposed $N = 5$) perturbed initial/boundary conditions
291 with perturbation magnitude ϵ (proposed $\epsilon = 10^{-3}$). If the coefficient of variation across ensemble
292 members exceeds $\tau_{\text{ens}} = 0.1$, flag. This is a model-free probe of sensitivity.

293 *Status:* these heuristics are specified but not yet implemented or validated. Testing on the
294 two known event-horizon failures (Euler buckling symmetry-breaking, critical-Reynolds branch
295 selection) and measuring the false-positive rate on the 132 interior- $\mathfrak{S}$ cases is required before
296 deployment. We report them as a concrete proposal, not a completed solution.

297 The efficiency ratio ρ

298 We define the setup-time efficiency ratio as $\rho = t_{\text{expert}}/t_{\text{framework}}$, where $t_{\text{framework}}$ is the wall-clock
299 time from natural-language input to first validated solution, and t_{expert} is the self-reported time
300 for a domain expert to produce a comparable result using standard tools. Both exclude compute
301 time (which is comparable). Expert times were collected from the 12 benchmark scientists
302 and the 3 independent replicators. The median $\rho = 480\times$ (Table 1); the interquartile range is
303 $360\times$ – $660\times$.

304 Adversarial test construction

305 50 inputs in three categories designed to probe $\partial\mathfrak{S}$: (a) 20 ambiguous/incomplete, targeting
306 condition (1); (b) 20 with subtle well-posedness issues, targeting conditions (2)–(3); (c) 10
307 where L^2 correctness diverges from qualitative correctness, targeting condition (4) near the event
308 horizon. Full list in Supplementary Table S3.

Implementation and reproducibility

Python. LLM backbone: Claude-3.5-Sonnet (Anthropic, version 20241022), temperature 0. Primitives: NumPy, SciPy, PyTorch. All code, `spec.md` files, cached inference logs, the 72 prospective tasks with expert baselines, and a Docker container are archived on Zenodo (DOI: 10.5281/zenodo.XXXXXXX; to be minted before submission) and at https://github.com/integritynoble/Physics_World_Model. Exact reproduction commands in `REPRODUCE.md`. The Docker container pins all dependency versions and reproduces the 12 development problems without network access.

Independent replication. Three researchers (J. Rodriguez, U. Michigan; A. Patel, Georgia Tech; M. Chen, Stanford) independently ran the 12 development problems on separate hardware with no additional instructions. All 12 produced bounded-error solutions within $\mathfrak{S}$. Variability: ± 0.2 dB (imaging), $\pm 3\%$ (PDE norms), $\pm 5\%$ (stochastic). Report in Supplementary Section S6.

Data and code availability

LoDoPaB-CT [12] (CC BY 4.0). Marmousi-2 [14] (public, SEG). GRI-Mech 3.0 [17] (public). Source code, `spec.md` files, cached LLM logs, and Docker container under MIT license.

Competing interests

C.Y. is the founder of NextGen PlatformAI C Corp, which develops the platform described here. Mitigation: (1) independent coordinator (Dr. M. Torres, ETH Zürich); (2) 12 external scientists with no company relationship; (3) all code, data, and logs publicly archived; (4) three independent researchers replicated results on separate hardware.

AI-use disclosure

The framework uses Claude-3.5-Sonnet (Anthropic) as the backbone for code generation and validation. The manuscript was drafted by the author with AI assistance for copy editing. All scientific claims, experimental design, and interpretation are solely the author’s.

Acknowledgements

We thank the 12 benchmark scientists (Supplementary Table S2; names to be disclosed with consent in the published version), the five blind-challenge participants, J. Rodriguez, A. Patel, and M. Chen for replication, and Dr. M. Torres for independent coordination.

Table 3. Extended Data Table 1: Controlled comparison on 12 development problems (all within $\mathfrak{S}$). Five conditions isolate the Judge’s contribution. “Correct”: solution within domain-specific tolerance. “Bound”: automated error bound provided. Each condition run $3\times$ with different seeds; majority vote reported. Note: 100% success on dev problems is expected, as these were used to design the Judge’s $\mathfrak{S}$ -verification checks.

	Full pipeline		No $\mathcal{A}_J$		GPT-4+spec		GPT-4 raw		PINN
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.
CT reconstruction	✓	✓	✓	—	✓	—	✓	—	✓
Marmousi FWI	✓	✓	—	—	—	—	—	—	—
GRI-Mech ignition	✓	✓	~	—	~	—	—	—	—
Granular flow	✓	✓	—	—	—	—	—	—	—
He ground state	✓	✓	~	—	~	—	~	—	—
BFS turbulent ^a	✓	✓	—	—	—	—	—	—	~
Topology opt.	✓	✓	✓	—	✓	—	✓	—	—
Waveguide	✓	✓	✓	—	✓	—	✓	—	✓
Heat equation	✓	✓	✓	—	✓	—	✓	—	✓
Fresnel diffraction	✓	✓	✓	—	✓	—	✓	—	✓
Rosby waves	✓	✓	~	—	~	—	~	—	—
React.-diffusion	✓	✓	~	—	~	—	~	—	~
Total	12	12	7	0	6	0	5	0	4

✓ = correct within tolerance. ~ = partially correct or qualitatively wrong. — = incorrect or not provided. PINN = DeepXDE [19], PDE problems only. ^aJHU Turbulence Database [18].

Table 4. Extended Data Table 2: Comparison with existing tools. Each excels in its domain; this work adds automated $\mathfrak{S}$ -verification to AI-generated code. Quantitative reference: on a standard Poisson problem ($\nabla^2 u = f$ on $[0, 1]^2$, 128^2 grid), framework error is 3.1×10^{-5} ; FEniCS achieves 2.8×10^{-5} ; both match the $O(h^2)$ theoretical rate.

	This work	COMSOL	FEniCS	OpenFOAM	PINNs	LLM agents
NL input	✓	—	—	—	—	✓
Cross-domain	$\mathfrak{S}$ tested	Multi-phys.	PDE	CFD	PDE	Partial
Error bound	✓	—	A post.	—	—	—
$\mathfrak{S}$ -verification	✓	—	—	—	—	—
No code required	✓	—	—	—	—	✓
ρ (median)	480×	—	—	—	—	—

A post. = a posteriori estimates for supported elements. LLM agents = Coscientist [6], ChemCrow [7]. ρ = setup-time efficiency ratio.

338 References

- 339 [1] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440,
340 2021.
- 341 [2] H. Tian et al. SciCode: a research coding benchmark curated by scientists. *arXiv:2407.13168*,
342 2024.
- 343 [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *J.*
344 *Comput. Phys.*, 378:686–707, 2019.
- 345 [4] A. Logg, K.-A. Mardal, and G. Wells (eds). *The FEniCS Book*. Springer, 2012.

- [5] F. Rathgeber et al. Firedrake: automating the finite element method. *ACM TOMS*, 43(3):1–27, 2016.
- [6] D. A. Boiko et al. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- [7] A. M. Bran et al. ChemCrow: augmenting LLMs with chemistry tools. *Nature Mach. Intell.*, 6:525–535, 2024.
- [8] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.
- [9] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9:267–293, 1956.
- [10] J. Hadamard. Sur les problèmes aux dérivées partielles. *Princeton Univ. Bull.*, 13:49–52, 1902.
- [11] COMSOL Multiphysics v. 6.2. COMSOL AB, Stockholm, 2024.
- [12] J. Leuschner et al. LoDoPaB-CT, a benchmark dataset for low-dose CT reconstruction. *Sci. Data*, 8:109, 2021.
- [13] W. van Aarle et al. The ASTRA Toolbox. *Ultramicroscopy*, 157:35–47, 2016.
- [14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade. *The Leading Edge*, 25:156–166, 2006.
- [15] C. Bunks et al. Multiscale seismic waveform inversion. *Geophysics*, 60:1457–1473, 1995.
- [16] S. Fomel et al. Madagascar: open-source software for reproducible experiments. *J. Open Res. Softw.*, 1:e8, 2013.
- [17] G. P. Smith et al. GRI-Mech 3.0. <http://combustion.berkeley.edu/gri-mech/>, 1999.
- [18] Y. Li et al. A public turbulence database cluster. *J. Turbulence*, 9:N31, 2008.
- [19] L. Lu et al. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.*, 63:208–228, 2021.
- [20] S. Boldo et al. Verified compilation of floating-point computations. *J. Autom. Reasoning*, 54:135–163, 2015.
- [21] H. Jasak, A. Jemcov, and Z. Tukovic. OpenFOAM: a C++ library for complex physics simulations. *Int. Workshop Coupled Meth. Numer. Dyn.*, 1000:1–20, 2007.